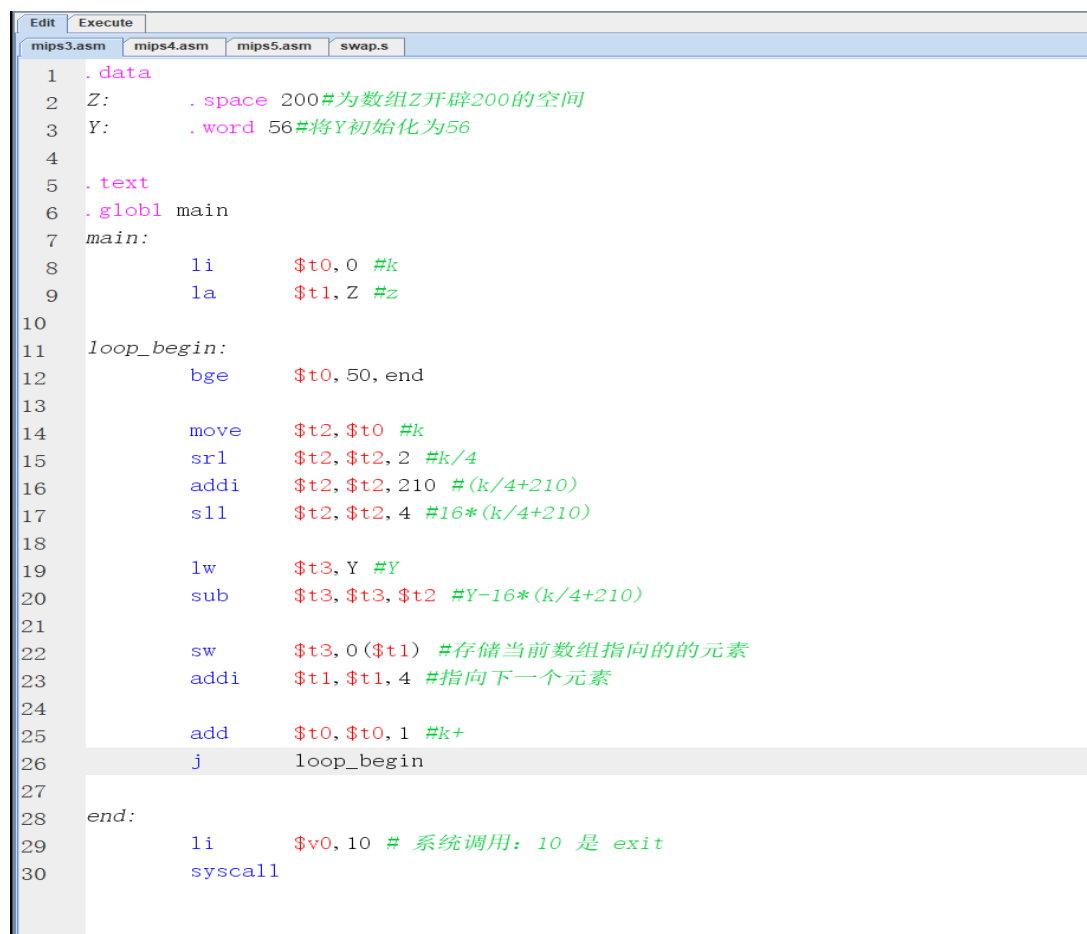


实验 1. 熟悉 MIPS 汇编程序开发环境，学习使用 MARs 工具。知道如何查看内存空间分配

一.用汇编程序实现以下伪代码：要求使用移位指令实现乘除法运算。

```
Int main ()
{
  Int K,Y;
  Int Z[50];
  Y=56;
  For(k=0;k<50;k++) Z[k]=Y-16*(k/4+210);
}
```

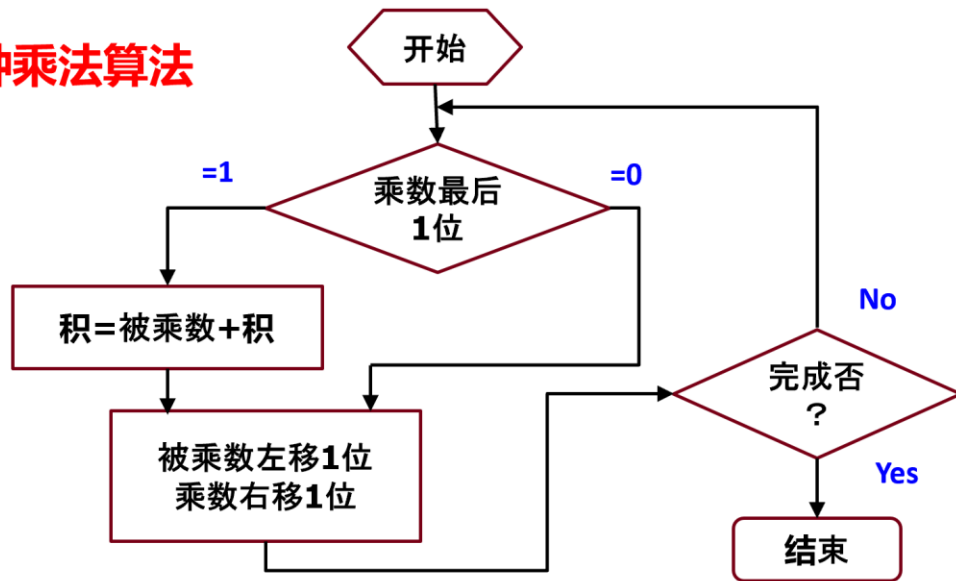


```
1  .data
2  Z:      .space 200 #为数组Z开辟200的空间
3  Y:      .word 56 #将Y初始化为56
4
5  .text
6  .globl main
7  main:
8      li    $t0, 0 #k
9      la    $t1, Z #z
10
11  loop_begin:
12      bge   $t0, 50, end
13
14      move  $t2, $t0 #k
15      srl   $t2, $t2, 2 #k/4
16      addi  $t2, $t2, 210 # (k/4+210)
17      sll   $t2, $t2, 4 #16*(k/4+210)
18
19      lw    $t3, Y #Y
20      sub   $t3, $t3, $t2 #Y-16*(k/4+210)
21
22      sw    $t3, 0($t1) #存储当前数组指向的的元素
23      addi  $t1, $t1, 4 #指向下一个元素
24
25      add   $t0, $t0, 1 #k+
26      j     loop_begin
27
28  end:
29      li    $v0, 10 # 系统调用: 10 是 exit
30      syscall
```

实验 2. 熟悉无符号乘法操作，并写出汇编代码 multu

乘数放在 \$a0 和 \$a1
结果放在 \$v0 核 \$v1 中

第一种乘法算法



```

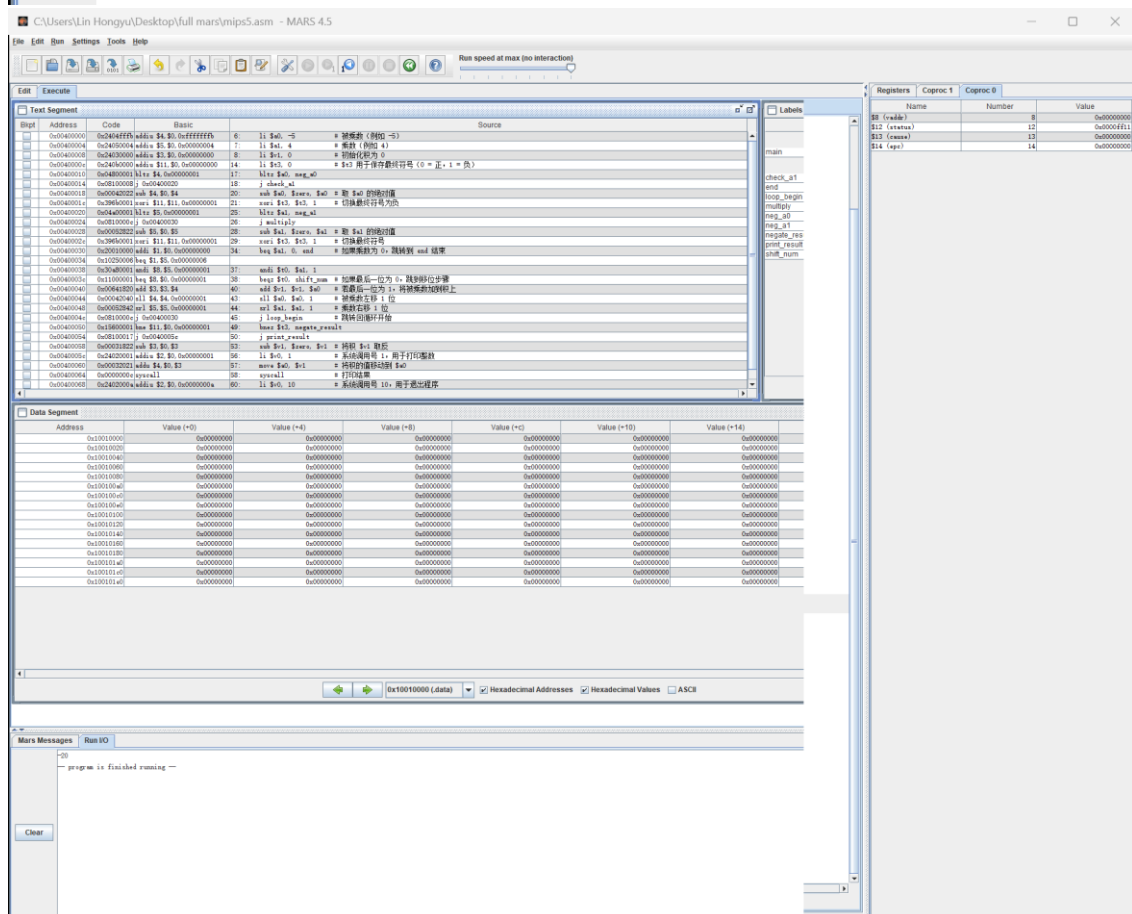
1  .data
2  .text
3  .globl main
4  main:
5      li    $a0, 5 #被乘数
6      li    $a1, 4 #乘数
7      li    $v1, 0
8
9  loop_begin:
10     beq    $a1, 0, end #乘数为0 结束
11
12     andi    $t0, $a1, 1 #取出乘数的最后一位
13     beqz    $t0, shift_num #最后一位为0 直接跳到移位步骤
14
15     add     $v1, $v1, $a0 #积=被乘数+积
16
17  shift_num:
18     sll     $a0, $a0, 1 #被乘数左移1
19     srl     $a1, $a1, 1 #乘数右移1
20     j       loop_begin
21
22  end:
23     li     $v0, 1 #系统调用号1 用于打印整数
24     move    $a0, $v1
25     syscall
26
27     li     $v0, 10
28     syscall
  
```



```

51
52 negate_result:
53     sub $v1, $zero, $v1 # 将积 $v1 取反
54
55 print_result:
56     li $v0, 1            # 系统调用号 1，用于打印整数
57     move $a0, $v1        # 将积的值移动到 $a0
58     syscall              # 打印结果
59
60     li $v0, 10           # 系统调用号 10，用于退出程序
61     syscall              # 退出程序
62

```



实验 3. 文件 [swap.s](#) 提供了一个调用你将编写的程序的代码模板。你可以在此程序上，添加你要写的 swap 代码，以方便测试。

```

void swap (int *px, int *py) {
    int temp;

    temp = *px;

    *px = *py;

    *py = temp;
}

```

}

编写汇编代码完成上述程序。由于所有 C 程序的局部变量都保存在栈中，因此变量 `temp` 也应保存在栈中 (未优化时的情况)。

换言之，不能使用 `$t0`（或者其它寄存器）来对应 `temp`。提示: 一共需要使用 6 条 `lw/sw` 指令。

如果允许使用 `$t0` 来保存 `temp` 变量，实现程序优化，本题可能会简单很多，本练习的部分目的是考查临时变量的栈存储。

完成后给老师解释你的代码。

```
26 swap:
27     #int temp;
28     addi $sp, $sp, -4          # 在栈上开辟空间，将栈指针 $sp 向下移动 4 字节
29
30     #temp = *px;
31     lw $t0, 0($a0) #存a0到temp栈中
32     sw $t0, 0($sp)
33
34     #*px = *py;
35     lw $t1, 0($a1) #存a1到a0中
36     sw $t1, 0($a0)
37
38     #*py = temp;
39     lw $t2, 0($sp)
40     sw $t2, 0($a1)
41
42     addi $sp, $sp, 4          # 恢复栈指针，将栈指针 $sp 向上移动 4 字节
43
44     jr $ra # 返回到调用点
```

